INTEGRATING BLOCKCHAIN AND OPPORTUNISTIC NETWORK TUTORIAL

I. Introduction

In this tutorial, we will walk through the process of integrating blockchain technology with an opportunistic network. This integration enhances security, trust, and data integrity in decentralized networks.

II. Prerequisites

- Basic understanding of opportunistic networks
- Familiarity with blockchain concepts
- A funded Tezos wallet on Ghostnet
- Installed tools:
  - Git
  - Python

1. Downloading and installing Mininet-WiFi:

Mininet-WiFi is a network emulator that supports the creation of wireless networks. Follow these steps to install it:

```
~$ git clone https://github.com/intrig-unicamp/mininet-wifi.git
~$ cd mininet-wifi
~/mininet-wifi$ sudo util/install.sh -Wlnfv6
~/mininet-wifi$ git pull
~/mininet-wifi$ sudo make install
```

Mininet-WiFi relies on Linux Kernel components to function properly. Of the different Linux distributions that can be used to this end, we recommend Ubuntu, since Mininet-WiFi was extensively tested on it.

2. Downloading and installing Pytezos

Pytezos is a Python library for interacting with Tezos blockchain, testing smart contracts, and writing Michelson scripts.

To install, run following command on Linux distributions:

```
~$ pip install pytezos
```

III. Implementation

1. File structure

Here is the link to main folder of the project: https://gitlab.com/herohthd/mininetwifi/-/tree/main/oppNet-blockchain?ref_type=heads

```
mininet-wifi/
├── README.md
```

```
├── oppnet/
│   ├── oppnet.py
├── contract/
│   ├── contract.py
├── pytezos/
│   ├── index.py
├── utils/
│   ├── main.py
│   └── infor.py
│   └── route.py
│   └── tezos.py
└── .gitignore
```

2. Set Up the Opportunistic Network

We created a python script named **oppnet.py** to set up the opportunistic network.

Import Necessary Modules

The script starts by importing necessary modules:

```
import random
import threading
import sys
import time
import threading
import math
import json
from mininet.log import setLogLevel, info
from mn_wifi.cli import CLI
from mn_wifi.net import Mininet_wifi
from mn_wifi.link import adhoc
from mn_wifi.mobility import Mobility
```

Create a Mininet-WiFi Network

The `main` function is where the network is set up and configured. Then we Set the logging level to 'info' to get detailed output about the process and finally we create an instance of Mininet-WiFi:

```
def main(num_hosts):
    setLogLevel('info')
    net = Mininet_wifi()
```

Create and config nodes (stations):

This section creates the nodes (stations) in the network with parameters name, ip address, power range, minimal x/y position, maximum x/y position.

```
info("*** Creating nodes\n")
    kwargs = {}
    if '-a' in sys.argv:
        kwargs['range'] = 100

    # Add stations based on the specified number of hosts
    stations = []
```

```
    for i in range(1, num_hosts + 1):
        station = net.addStation(f'sta{i}', ip=f'10.0.0.{i}/8',
                                 min_x=random.randint(0, 100),
                                 max_x=random.randint(100, 200),
                                 min_y=random.randint(0, 100),
                                 max_y=random.randint(100, 200),
                                 min_v=5, max_v=10)
        stations.append(station)

    info("*** Configuring nodes\n")
    net.configureNodes()
```

Create controller

Add a controller to the network:

```
    c1 = net.addController('c1')
```

Configuring propagation model:

Set the propagation model to 'logDistance' with an exponent of 4.5:

```
    info("*** Configuring propagation model\n")
    net.setPropagationModel(model="logDistance", exp=4.5)
```

Plot Graph (Optional)

Plot the network graph if the -p argument is not provided:

```
    if '-p' not in sys.argv:
        net.plotGraph()
```

Set Mobility Model

Set the mobility model for the stations to 'RandomDirection':

```
    net.setMobilityModel(time=0, model='RandomDirection',
                         max_x=250, max_y=250, seed=20)
```

Create Links Between Stations

Create ad-hoc links between the stations:

```
    info("*** Associating and Creating links\n")
    # Add links between stations
    for i in range(len(stations)):
        for j in range(i + 1, len(stations)):
            net.addLink(stations[i], stations[j], cls=adhoc, intf=f'sta{i + 1}-wlan0',
                        ssid='adhocNet', mode='g', channel=5, **kwargs)
```

Start the Network

Build and start the network:

```
info("*** Starting network\n")
net.build()
```

3. Set Up the Smart Contract

In this part, we will walk through a SmartPy smart contract designed to store and manage network information. We will explain the structure and functionality of the contract and guide you on how to deploy and interact with it.

a) Overview of the Smart Contract

The provided smart contract is written in SmartPy, a Python-like language used to write smart contracts for the Tezos blockchain. This contract has two main functionalities:

1. Storing distance information between network stations.
2. Storing and updating network information from different stations.

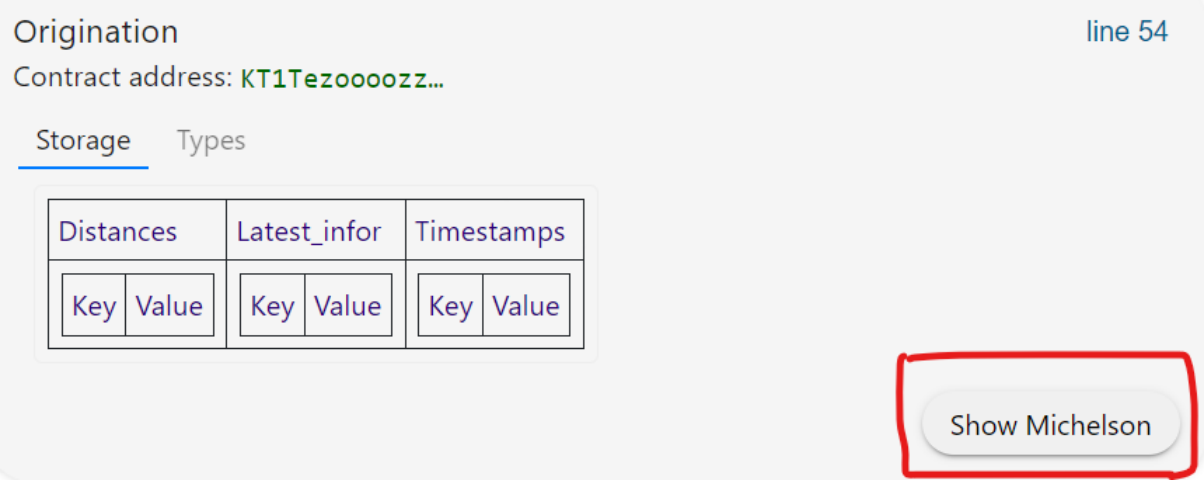
To implement and test the contract, we recommend using Smartpy IDE, an intuitive and powerful smart contract development IDE for Tezos blockchain.

b) The Smart Contract Code

c) Explanation of the Contract

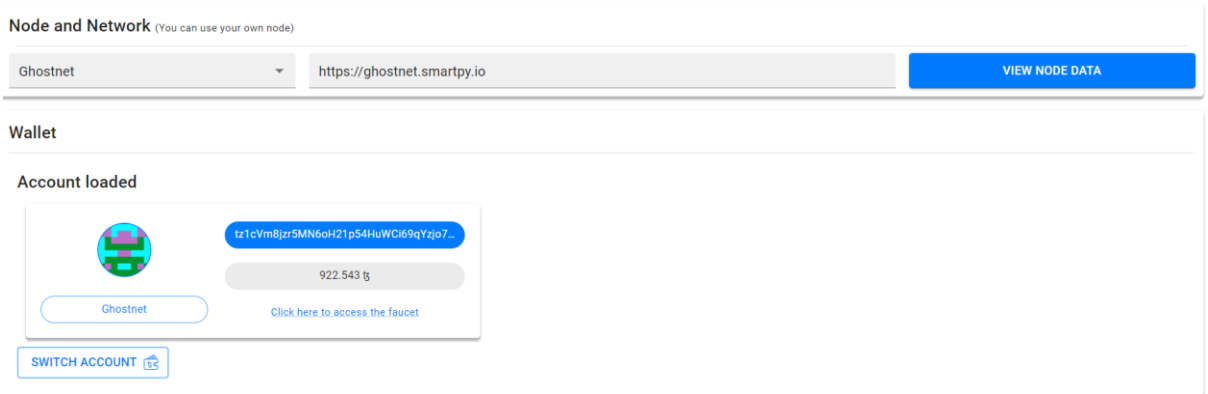
d) Deploy the contract on testnet

- Step 1: Click on **Show Michelson**

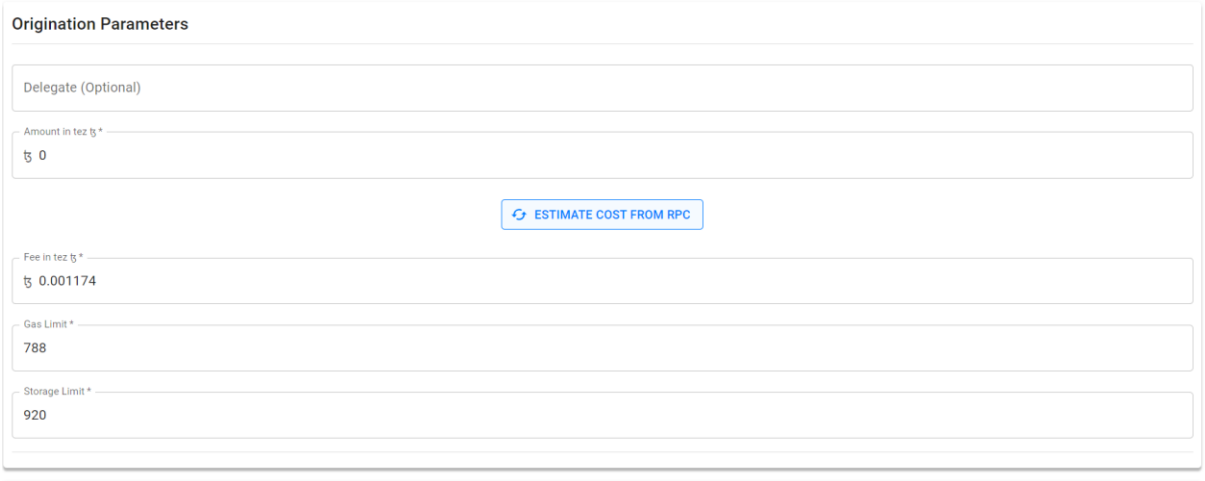


- Step 2: After that, it'll open a modal. Then click on **Deploy Contract.**

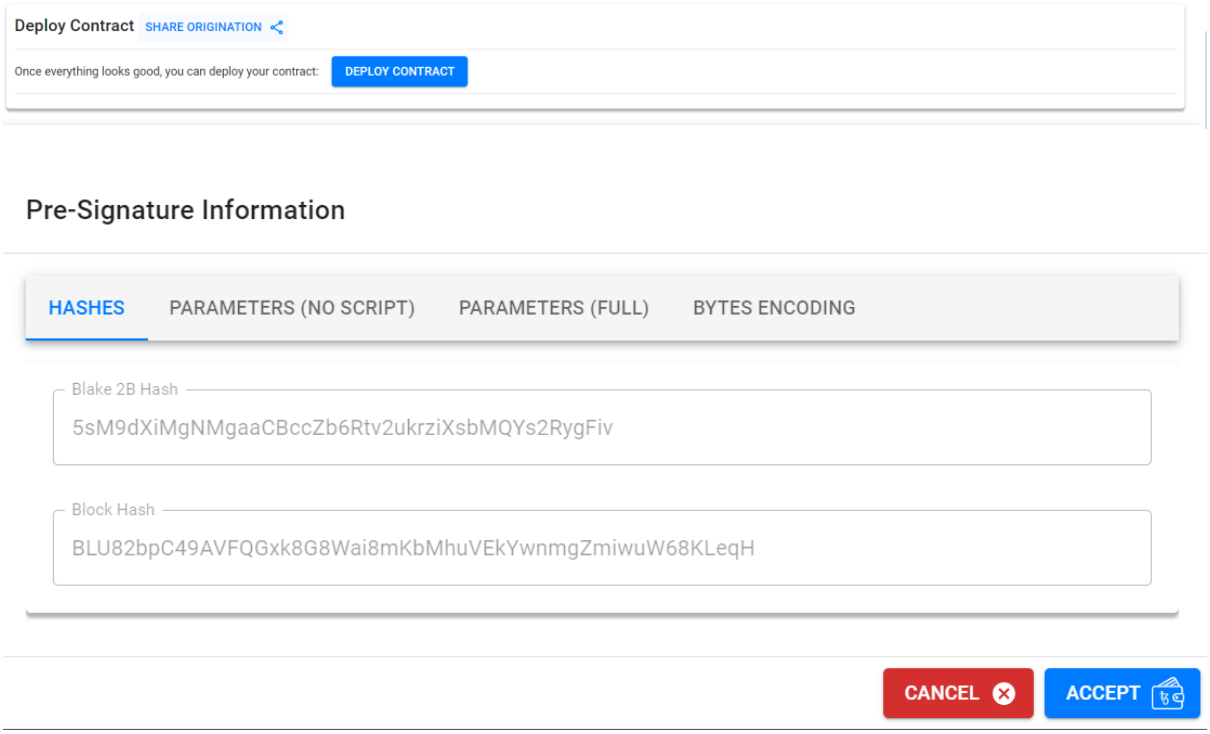
- Step 3: On deploying tab, select Ghostnet and connect your account.



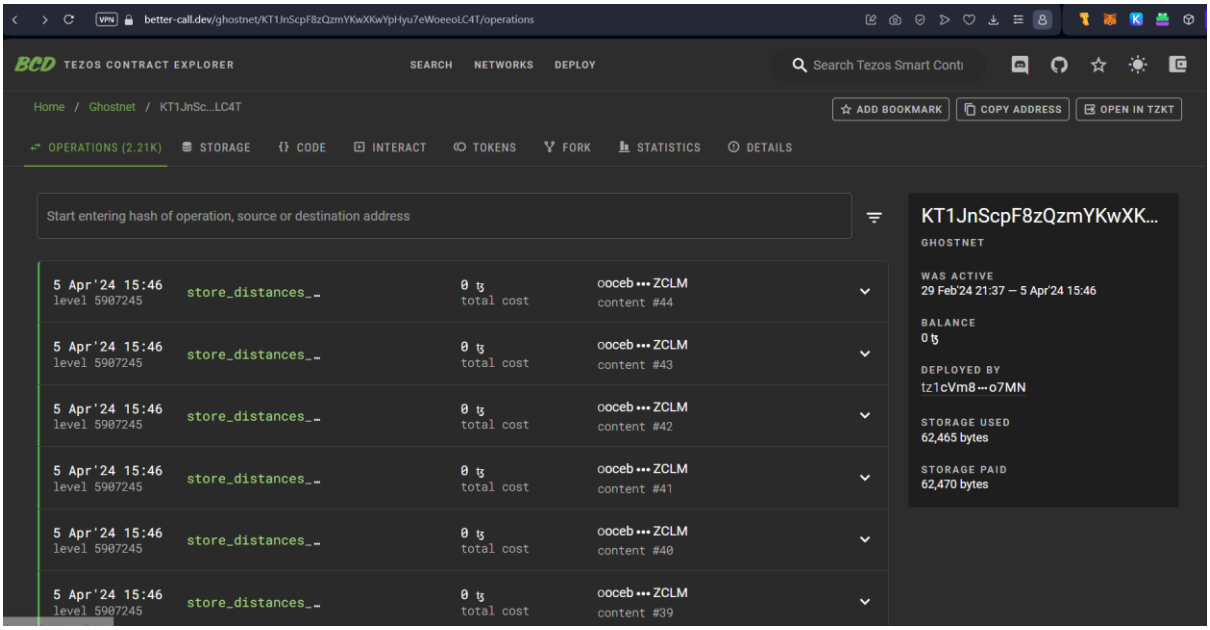
- Step 4: Then click **ESTIMATE COST FROM RPC** button



- Step 5: Sign transaction to deploy contract



After deploying, you should have a contract on blockchain which you can interact with via Better-call-dev, a Tezos smart contract explorer and search engine. Here's example of a deployed contract:



4. Integrate Opportunistic Network with Smart contract

Get network information

In the file **oppnet.py**, we create another function and assign it to a thread to keep track of network information.

Purpose:

The `track_network_information` function is designed to continuously monitor and record the network information of the stations in the Mininet-WiFi simulation. It gathers data on the current status and position of each station, as well as the distances between all pairs of stations. This information is then saved into a JSON file for further analysis or use.

How It Works:

1. **Initialization:**
    - The function starts by determining which stations are currently online (i.e., their interfaces are up) and initializes an infinite loop to continuously collect network data.
2. **Timestamp Generation:**
    - At each iteration of the loop, a timestamp is generated to mark the exact time when the data was collected.
3. **Network Information Collection:**
    - A dictionary named `network_info` is created to store the collected data. It includes the timestamp, a nested dictionary of station information, and a list of distances between stations.
    - The function iterates over all the stations in the network to gather their current information, such as position, IP address, and other relevant details. This information is stored in the `network_info` dictionary under the key corresponding to each station's name.
4. **Distance Calculation:**
    - The function calculates the distances between all pairs of stations. To avoid redundant calculations, it uses a set named `recorded_distances` to keep track of already computed distance pairs.
    - For each unique pair of stations, the distance is calculated using the `calculate_distance` function, and the result is appended to the network_info dictionary.
5. **Writing to JSON File:**
    - The collected network information is written to a JSON file specified by `file_path`. This allows the data to be saved in a structured format that can be easily accessed and analyzed later.
6. **Signal File Creation:**
    - A signal file named "simulation_complete.signal" is created to indicate that the network information has been successfully recorded.
7. **Sleep Interval:**
    - The function then sleeps for 60 seconds (1 minute) before repeating the process, ensuring that the network information is updated at regular intervals.

```
def track_network_information(net, file_path):
```

```
    signal_file_path = "simulation_complete.signal"

    online_stations = [sta for sta in net.stations if sta.intf().isUp() == True]
    while True:
        timestamp = time.strftime('%Y-%m-%d %H:%M:%S', time.localtime())
        network_info = {"timestamp": timestamp, "stations": {}, "distances": []}
        recorded_distances = set()

        for sta in net.stations:
            station_info = track_station_information(sta, online_stations)
            network_info["stations"][sta.name] = station_info

        for sta1 in net.stations:
            for sta2 in net.stations:
                if sta1 != sta2:
                    distance_pair = frozenset({sta1.name, sta2.name})
                    if distance_pair not in recorded_distances:
                        distance = calculate_distance(sta1, sta2)
                        network_info["distances"].append({"staX": sta1.name, "staY": sta2.name,
"distance": distance})
                        recorded_distances.add(distance_pair)

        with open(file_path, "w") as json_file:
            json.dump(network_info, json_file, indent=2)

        with open(signal_file_path, "w"):
            pass

        time.sleep(60)
```

Store network information on the contract

This server script is designed to read network information from a JSON file and push it to a Tezos smart contract. The script runs continuously, listening for changes in the network information file and updating the smart contract whenever new information is available.

Key Components:

1. **Environment Setup:**
   - The script uses environment variables stored in a `.env` file for sensitive information like the Tezos key (PYTEZOS_KEY).
   - It also sets up the Tezos node URL and the smart contract address.
2. **Logging:**
   - Logging is configured to provide information about the server's operations and any errors that occur.
3. **Function Definitions:**
   - `read_network_information(file_path)`: Reads and returns the content of the JSON file specified by file_path.
   - `update_network_infor (network_info, contract)`: Updates the smart contract with the network information. It extracts timestamp, stations, and

distances from the network info and sends this data to the smart contract using Tezos operations.
    - `listen_for_changes(file_path, contract)`: Continuously listens for changes in the network_information.json file. When a change is detected, it reads the file, updates the smart contract, and deletes the signal file to indicate that the update has been processed.
4. **Main Function:**
    - `main()`: Initializes the server, loads the smart contract, and starts a thread to listen for changes in the network information file.

How It Works:

1. **Environment Variables:**
    - The script loads the environment variables from a `.env` file using the python-dotenv library. This includes the Tezos private key required to sign transactions.
2. **Contract Initialization:**
    - The script connects to the Tezos network using the node URL and accesses the smart contract using the provided contract address.
3. **Reading Network Information:**
    - The `read_network_information` function reads the JSON file that contains the network information. This file is updated by the simulation periodically.
4. **Updating the Smart Contract:**
    - The `update_network_infor` function extracts relevant information from the network info and prepares Tezos operations to update the smart contract. It handles two types of information:
        - **Station Information:** Includes details like position, frequency, mode, transmission power, range, antenna gain, and status (online/offline).
        - **Distance Information:** Includes distances between pairs of stations.
    - These operations are then sent to the Tezos blockchain in a bulk transaction to ensure atomicity and efficiency.
5. **Listening for Changes:**
    - The `listen_for_changes` function continuously checks for the presence of a signal file (simulation_complete.signal). This file indicates that new network information is available.
    - When the signal file is detected, it reads the network information from the JSON file, updates the smart contract, and deletes the signal file.
6. **Server Execution:**
    - The `main` function sets up the logging, initializes the connection to the smart contract, and starts a thread to listen for changes in the network information file.
    - The main thread keeps running to ensure the server remains active.

```
import os
import time
import json
from dotenv import load_dotenv
from pytezos import pytezos
from threading import Thread
import logging
from datetime import datetime

# Load environment variables from .env
```

```
load_dotenv()

# Fetch the key from the environment variable
PYTEZOS_KEY = os.environ["PYTEZOS_KEY"]

if not PYTEZOS_KEY:
    raise ValueError("PYTEZOS_KEY environment variable is not set")

CONTRACT_ADDRESS = "KT1JnScpF8zQzmYKwXKwYpHyu7eWoeeoLC4T"
CONTRACT_STORAGE = {"timestamps": {}, "latest_info": {}, "distances": {}}
NODE_URL = "https://ghostnet.ecadinfra.com"

# Configure logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def read_network_information(file_path):
    with open(file_path, "r") as json_file:
        return json.load(json_file)

def update_network_infor (network_info, contract):
    global CONTRACT_STORAGE

    timestamp = network_info.get("timestamp")
    stations = network_info.get("stations", {})
    distances = network_info.get("distances", {})

    stored_distances = []

    # Create a list of OperationGroup objects for store_network_info calls
    stored_network_info = [
        contract.store_network_info(
            timestamp=timestamp,
            station_name=station_name,
            network_info={
                "position": ",".join(map(str, station_info["coordination"])),
                'freq': str(station_info["frequency"]),
                'mode': str(station_info["mode"]),
                'tx_power': str(station_info["tx_power"]),
                'range': str(station_info["range"]),
                'antenna_gain': str(station_info["antenna_gain"]),
                'status': 'online' if station_info["status"] == "online" else "offline"
            }
        )
        for station_name, station_info in stations.items()
    ]

    # Create a list of OperationGroup objects for store_distances_info calls
```

```
    stored_distances = [
        contract.store_distances_info(
            staX=distance_info["staX"],
            staY=distance_info["staY"],
            distance=str(distance_info["distance"])
        )
        for distance_info in distances
    ]

    # Perform bulk operation for store_network_info and store_distances_info calls
    pytezos.bulk(*stored_network_info, *stored_distances).send(min_confirmations=1)
    logger.info(f"{datetime.now().strftime('%d-%m-%Y %H:%M:%S')} Bulk operation for
store_network_info and store_distances_info completed")

def listen_for_changes(file_path, contract):
    signal_file_path = "simulation_complete.signal"

    while True:
        # Wait until the signal file is created by the simulator
        while not os.path.exists(signal_file_path):
            time.sleep(1)

        try:
            # Read the content of the JSON file
            network_info = read_network_information(file_path)

            # Update the contract with the new network information
            update_network_infor(network_info, contract)

        except Exception as e:
            logger.error(f"Error: {e}")

        # Remove the signal file to indicate the content has been read
        os.remove(signal_file_path)

def main():
    logger.info(f"{datetime.now().strftime('%d-%m-%Y %H:%M:%S')} Server is running")
    # Load the existing contract using the provided CONTRACT_ADDRESS
    py = pytezos.using(shell=NODE_URL)
    contract = py.contract(CONTRACT_ADDRESS)

    # Start a thread to listen for changes in the network_information.json file
    file_path = "network_information.json"
    update_thread = Thread(target=listen_for_changes, args=(file_path, contract))
    update_thread.start()

    # Keep the main thread alive
    while True:
```

```
        time.sleep(1)

if __name__ == "__main__":
    main()
```

IV. Running

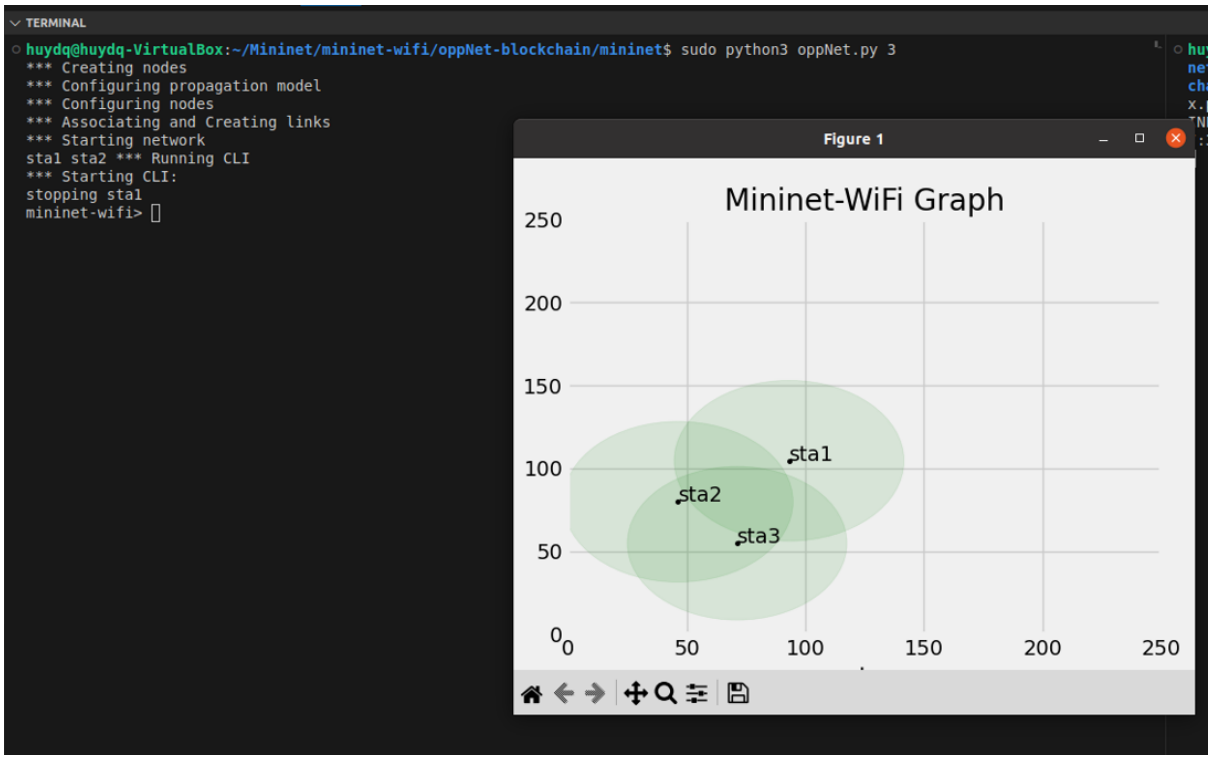
To run the project, first on one terminal, we run the server:

```
~/mininet-wifi/ $ python3.8 pytezos/index.py
```

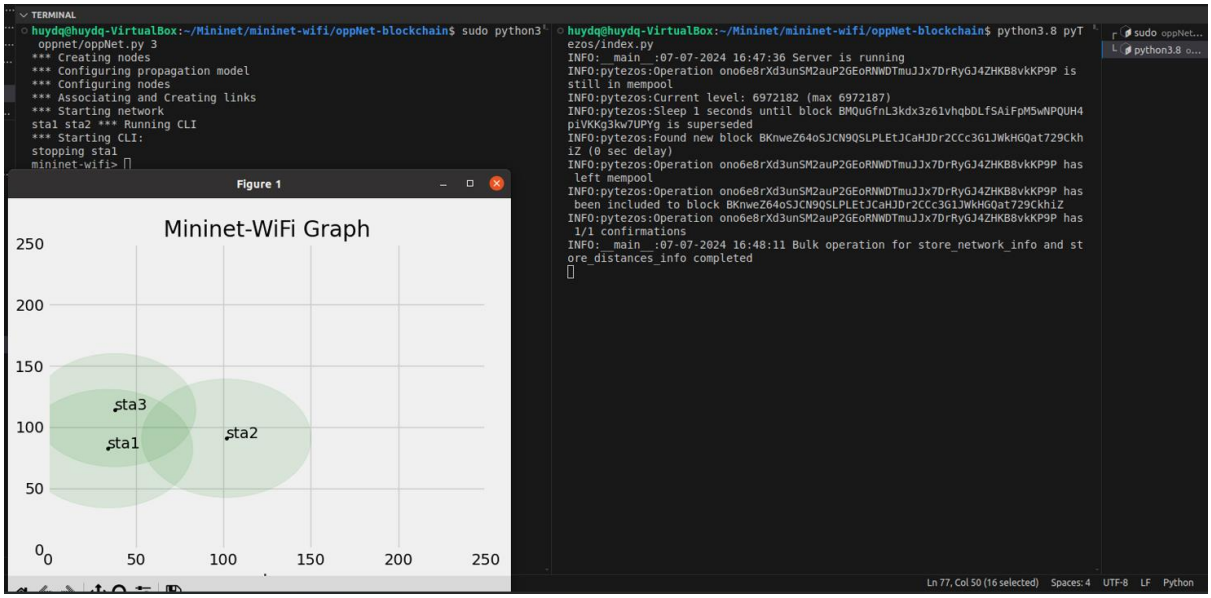
The server will start and listen to the updating network information signal:

After that, on other terminal, we run the opportunistic network environment with updating network information thread.

```
~/mininet-wifi$ sudo python3 oppnet/oppnet.py
```



And finally, we have a running opportunistic network environment with a server to push latest network information to the smart contract.



You can track the operation and current storage of the smart contract on [better-call-dev](better-call.dev/ghostnet/KT1JnScpF8zQzmYKwXKwYpHyu7eWoeeoLC4T/operations) as well.